

Evolution of Major Programming Languages

Concepts of Programming Languages

- Robert W. Sebesta

Course: SY CSE

Introduction

- Programming languages evolved to simplify writing software.
- Each generation added abstraction, readability, and power.
- We explore major languages and their contributions.

- **A. Pseudocode**
- **Definition:** Informal high-level language that describes the logic of an algorithm
- **Not executed by a computer**
- **Purpose:** Design and planning
- **Example:**
- Start
- Input A, B
- $\text{Sum} \leftarrow A + B$
- Print Sum
- End

- **1. Machine and Assembly Languages (1940s–1950s)**
- **Machine Language**
- **Binary language** specific to each CPU.
- Difficult to read, write, and debug.
- No abstraction — programmer must manage memory and CPU directly.
- ♦ *Example:*
- 10111000 00000000 (x86 instruction to move value into a register)
- **Assembly Language**
- **Symbolic representation** of machine instructions.
- Uses **mnemonics** (e.g., MOV, ADD, SUB).
- Easier than machine code, but still hardware-specific.
- ♦ *Example:*
- MOV AX, 5
- ADD AX, 3
- **Contribution:** Laid foundation for understanding low-level operations and instruction cycles.

- **B. Assembly Language**
- **Low-level language:** Uses mnemonics instead of binary
- **Machine-specific**
- Requires **Assembler** to convert into machine code
-  **Example (Intel x86 assembly):**
- MOV AX, 5
- MOV BX, 10
- ADD AX, BX

FORTRAN (1957)

- First high-level language for scientific computing
- Supports loops, conditions, arrays
- Example: `DO 10 I = 1, 10`
- Contribution: Efficient numeric computation

- **2. FORTRAN (Formula Translation) – 1957**
- **First high-level programming language** designed for **scientific and engineering** computations.
- Introduced **control structures** like loops, conditionals, and functions.
- Allowed algebraic-style syntax.
-  *Example:*
- DO 10 I = 1, 10
- SUM = SUM + I
- 10 CONTINUE
- **Contribution:** Enabled efficient numeric computation; foundation of scientific computing.

COBOL (1959)

- For business applications
- English-like syntax
- Example: IF TOTAL > 10000 DISPLAY "BONUS"
- Contribution: Structured data handling

- **3. COBOL (Common Business-Oriented Language) – 1959**
- Designed for **business applications**: record keeping, accounting, payroll.
- Emphasized **readability, English-like syntax, and file processing**.
-  *Example:*
- IF TOTAL-SALES > 10000
- DISPLAY 'BONUS GRANTED'.
- **Contribution**: Still used in legacy business systems; pioneered structured data handling.

LISP (1958)

- Used in AI and symbolic processing
- Functional paradigm, recursion
- Example: (defun fact (n) ...)
- Contribution: Introduced garbage collection

- **4. LISP (LISt Processing) – 1958**
- Created for **artificial intelligence** applications.
- Based on **functional programming** and **recursion**.
- Everything is a **list** or function.
-  *Example:*
- (defun factorial (n)
- (if (= n 0) 1 (* n (factorial (- n 1)))))
- **Contribution:** Introduced garbage collection and recursion; inspired modern functional languages.

ALGOL (1960)

- Universal language for algorithms
- Introduced block structure and scope
- Influenced Pascal, C
- Example: `begin integer a; a := 5; end`

- **5. ALGOL (Algorithmic Language) – 1960**
- Designed as a **universal language** for publishing algorithms.
- Introduced **block structures, scope, and structured programming**.
- Syntax heavily influenced **Pascal, C, and Java**.
- ♦ *Example:*
- begin
- integer a, b;
- a := 10;
- b := a * 2
- end
- **Contribution:** Laid the groundwork for many modern languages; used in academic research.

BASIC (1964)

- For education and beginners
- Simple and interactive
- Example: `10 PRINT "HELLO"`
- Contribution: Widespread adoption in schools

- **6. BASIC (Beginner's All-purpose Symbolic Instruction Code) – 1964**
- Designed for **education** and **ease of use**.
- Simplified syntax for **non-technical users**.
-  *Example:*
- 10 PRINT "HELLO"
- 20 GOTO 10
- **Contribution:** Made programming accessible to schools and hobbyists

Pascal (1970)

- Structured programming and teaching
- Strong typing
- Example: for $i := 1$ to 10 do
- Contribution: Educational use, good design

- **7. Pascal – 1970**
- Designed for **teaching structured programming** and data structuring.
- Emphasized **type checking, clarity, and structured control flow.**
- ♦ *Example:*
- for i := 1 to 10 do
- writeln(i);
- **Contribution:** Widely used in education; foundation for later system languages.

C (1972)

- System programming language
- Low-level memory access
- Example: `for(int i=0; i<10; i++)`
- Contribution: Influenced many modern languages

- **8. C – 1972**
- Designed for **system programming** (especially UNIX).
- Offers **low-level memory access** with **high-level control structures**.
- Portable and powerful.
- ♦ *Example:*
- ```
for (int i = 0; i < 10; i++) {
```
- ```
    printf("%d\n", i);
```
- ```
}
```
- **Contribution:** Most widely used systems language; influenced C++, Java, Python, etc.

# C++ (1983)

- Extension of C with OOP
- Classes, inheritance
- Example: `class Person { public: void greet(); };`
- Contribution: Large-scale systems development

- **9. C++ – 1983**
- **Extension of C with Object-Oriented Programming (OOP).**
- Supports **encapsulation, inheritance, polymorphism.**
-  *Example:*
- `class Person {`
- `public:`
- `void greet() {`
- `cout << "Hello";`
- `}`
- `};`
- **Contribution:** Dominated systems and application development in the 1990s.

# Java (1995)

- Platform-independent via JVM
- Fully object-oriented
- Example: `public class Hello { ... }`
- Contribution: Enterprise and Android development

- **10. Java – 1995**
- Designed to be **platform-independent** using the **Java Virtual Machine (JVM)**.
- Strong emphasis on **OOP, security, and networked applications**.
- ♦ *Example:*
- public class Hello {
- public static void main(String[] args) {
- System.out.println("Hello World");
- }
- }

**Contribution:** Foundation of Android development; still widely used in enterprise systems.

# Python (1991)

- Simple syntax, powerful libraries
- Multi-paradigm: procedural, OOP, functional
- Example: `for i in range(10): print(i)`
- Contribution: Popular in education, AI, web, data science



- **11. Python – 1991 (Popular post-2000s)**
- Focused on **simplicity, readability, and minimal syntax.**
- Supports **multiple paradigms**: object-oriented, functional, procedural.
-  *Example:*
- `for i in range(10):`
- `print(i)`
- **Contribution:** Dominates web, AI, data science, education; known for rapid development.

# Summary Table

| Language       | Year | Domain     | Notable Features              |
|----------------|------|------------|-------------------------------|
| <b>FORTRAN</b> | 1957 | Scientific | Loops, arrays                 |
| <b>COBOL</b>   | 1959 | Business   | File handling, English syntax |
| <b>LISP</b>    | 1958 | AI         | Functional, recursion         |
| <b>ALGOL</b>   | 1960 | General    | Structured programming        |
| <b>BASIC</b>   | 1964 | Education  | Simplicity                    |
| <b>Pascal</b>  | 1970 | Education  | Type safety, teaching         |
| <b>C</b>       | 1972 | System     | Low-level + high-level mix    |
| <b>C++</b>     | 1983 | General    | Object-oriented               |
| <b>Java</b>    | 1995 | Enterprise | JVM, portability              |
| <b>Python</b>  | 1991 | General    | Easy syntax, multi-paradigm   |

# Conclusion

- Evolution driven by domain needs and abstraction
- Each language contributed unique ideas
- Foundation for modern programming